

[CS230] Sequence Models: Beam Search (Optimization)

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1-10.8 RNN, Embedding, Seq2Seq - *Completed*
- **10.9 Beam Search**
 - * Greedy Search vs Beam Search
 - * Beam Width Parameter (B)
 - * Refinements: Log Likelihood & Length Normalization

Chapter 11. Attention Mechanism - *Next Part*

지난 시간 복습 및 연결

우리는 Seq2Seq 모델로 번역기를 만들었습니다. 그런데 막상 돌려보니 엉뚱한 문장이 나옵니다. 이유는 우리가 '당장의 최선(Greedy)'만 선택했기 때문입니다. 1등만 뽑아서 문장을 이었더니 전체 문맥은 엉망이 된 것이죠. 인생과 마찬가지로, "당장의 최선이 결과적인 최선은 아닐 수" 있습니다. 여러 가능성을 동시에 탐색하는 **빔 서치(Beam Search)**가 필요합니다.

1 Unit Overview

□ 핵심 목표

이 단원은 텍스트 생성 모델의 성능을 결정짓는 탐색 알고리즘을 다룹니다.

- **Greedy:** 매 순간 1등만 뽑는 방식의 한계(Local Optima)를 이해합니다.
- **Beam:** 상위 B 개의 가능성을 살려두며 탐색하는 원리를 파악합니다.
- **Refinement:** 로그 우도(Log Likelihood)와 길이 정규화(Length Normalization)를 통해 수학적 안정성을 확보합니다.

2 Essential Terminology

용어	의미	특징
Greedy Search	매번 확률 1등 단어만 선택.	빠르지만 최적해 보장 못함.
Beam Search	매번 상위 B 개를 유지하며 탐색.	느리지만 더 좋은 문장 생성.
Beam Width (B)	유지할 후보 개수 (보통 3-10).	$B = 1$ 이면 Greedy와 같음.

3 Core Concepts: 여러 길을 동시에 가라

3.1 1. The Problem: Greedy Search

매 스텝에서 확률이 가장 높은 단어 1개만 고르고 다음으로 넘어갑니다.

□ 미로 찾기 (직관적 비유)

갈림길에서 무조건 "출구와 가까워 보이는 쪽"으로만 가는 것과 같습니다. 가다 보니 막다른 길일 수 있지만, 되돌아갈 수 없습니다(No Backtracking).

3.2 2. The Solution: Beam Search

매 순간 상위 B 개의 가능성(Hypothesis)을 유지합니다. ($B = 3$ 가정)

- **Step 1:** 첫 단어 후보 중 상위 3개를 뽑습니다. (예: "in", "the", "a")
- **Step 2:** 살아남은 3개의 단어 각각에 대해, 다음 단어 확률을 계산합니다.
- **Step 3:** 총 30,000 개 ($3 \times$ 단어장) 조합 중 다시 상위 3등까지만 남기고 나머지는 버립니다.
- **Repeat:** 문장이 끝날 때까지 반복합니다.

4 Deep Dive: Refinements (정밀화)

단순 확률 곱셈에는 수학적 문제가 있습니다. 이를 보정해야 합니다.

4.1 1. Log Likelihood (로그 우도)

확률(0.1, 0.05...)을 수십 번 곱하면 값이 0.0000... 이 되어 컴퓨터가 0으로 인식해버립니다 (Underflow). 해결책은 로그(log)를 취하는 것입니다. 곱셈이 덧셈으로 변합니다.

□ Score Function (핵심 수식)

$$\sum_{t=1}^{T_y} \log P(y^{(t)} | y^{(1...t-1)}, x)$$

로그 합을 최대화하는 것은 확률 곱을 최대화하는 것과 수학적으로 동일합니다.

4.2 2. Length Normalization (길이 정규화)

로그 확률(음수)을 계속 더하면, 문장이 길어질수록 점수는 계속 낮아집니다. 모델이 "짧은 문장"을 과도하게 선호하게 됩니다. 해결책은 점수를 문장 길이(T_y)로 나누어 평균을 내는 것입니다.

□ Normalized Score (핵심 수식)

$$\text{Score} = \frac{1}{T_y^\alpha} \sum \log P(\dots)$$

- α : 보정 계수 (보통 0.7).
- 긴 문장에 대한 페널티를 완화해줍니다.

5 FAQ & Pitfalls

Q. B 를 무조건 크게 하면 좋은가요?

A. 아닙니다. B 가 크면 더 좋은 문장을 찾을 확률은 높지만, 계산량과 메모리가 B 배로 늘어납니다. 실무에서는 보통 $B = 3 \sim 10$ 정도를 쓰며, 아주 정밀한 번역이 필요할 때만 더 키웁니다.

Q. 빔 서치는 최적해(Global Optima)를 보장하나요?

A. 아니요. B 가 무한대(BFS)가 아닌 이상, 휴리스틱 탐색이므로 완벽한 최적해를 보장하지 않습니다. 하지만 Greedy보다는 훨씬 낫습니다.

다음 단계 (Next Step)

빔 서치로 문장 생성 능력은 좋아졌습니다. 하지만 여전히 근본적인 문제가 남았습니다. Seq2Seq의 인코더는 아무리 긴 문장이라도 하나의 고정된 벡터(Context Vector)에 요약넣어야 한다는 '병목(Bottleneck)' 현상입니다.

"번역할 때 문장 전체를 외우고 하는 사람은 없습니다. 필요한 단어를 그때그때 다시 보면서(Attention) 하죠." 다음 시간에는 딥러닝 역사상 가장 위대한 발명 중 하나인 [Attention Mechanism]을 통해 이 병목을 부수고 Transformer로 가는 문을 열겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Greedy:** 빠르지만 시야가 좁아 최적 문장을 놓친다.
2. **Beam Search:** B 개의 유망주를 끝까지 키운다.
3. **Log:** 언더플로우 방지를 위해 확률 곱 대신 로그 합을 쓴다.
4. **Normalize:** 짧은 문장 편향을 막기 위해 길이로 나눈다.